School Website Management System

SOFTWARE DESIGN DOCUMENT (SDD)

Department of Information Technology | Faculty of Computing | IUB

Student Name:	Esha Zafar
Roll No:	4064
Submitted To:	Sir Abdul Karim Nawaz
Date:	June 10, 2026
Department:	Information Technology
Faculty:	Faculty of Computing
University:	The Islamia University of Bahawalpur
Version:	2.0 (Expanded Edition)

ACADEMIC SUBMISSION | DEPARTMENT OF IT | IUB | 2026

♦ Table of Contents

1 Introduction & Document Purpose

1.1 Purpose

This Software Design Document (SDD) provides the complete technical blueprint for the School Website Management System (SWMS) — a web-based platform developed for the Department of Information Technology, Faculty of Computing, The Islamia University of Bahawalpur. It describes architectural decisions, data models, UI layouts, security measures, and non-functional requirements that govern the system.

1.2 Scope

- Centralised web portal for Administrators, Teachers, Students, and Parents.
- Student registration, academic results, attendance tracking, and notice board.
- Role-Based Access Control (RBAC) ensuring data privacy for each stakeholder.
- Downloadable PDF reports for result cards and attendance summaries.
- Fully responsive design: desktop, tablet, and mobile browsers.
- Secure HTTPS communication with encrypted credentials.
- RESTful API layer for future mobile application integration.
- Automated email/SMS notifications for results and attendance alerts.
- Bulk data import/export via CSV and Excel formats.
- Comprehensive audit trail for all administrative actions.

1.3 Intended Audience

Role	Interest	Sections
Instructor / Evaluator	Academic assessment	All sections
System Architect	Design validation	2, 3, 7, 8
Frontend Developer	UI implementation	4, 7
Backend Developer	Logic & API	2, 3, 7, 8
QA Engineer	Testing plan	6, 9
DevOps Engineer	Deployment	5, 10
Project Manager	Risk & planning	6, 11

1.4 Document Conventions

Bold text highlights key terms. SQL blocks appear in monospaced Courier font. Primary keys are labelled PK and foreign keys FK. Security control IDs follow SEC-nn format. API endpoints are prefixed with HTTP method (GET, POST, PUT, DELETE).

2 Application Architecture

2.1 System Overview

SWMS employs a classic Three-Tier Client-Server Architecture separating presentation, business logic, and data concerns. Each tier communicates only with its adjacent tier, maximising modularity. All traffic is served over HTTPS / TLS 1.3 via Nginx.

Tier	Name	Key Components	Responsibility
1	Presentation	HTML5, CSS3, JS, Bootstrap 5	Render UI; capture user input
2	Business Logic	PHP 8.2 / Laravel 10 (MVC)	Validation, rules, API routing, ORM
3	Data	MySQL 8.0 + Redis Cache	Persistent storage, caching, backups

2.2 Architecture Diagram

Figure 1 — Three-Tier Application Architecture

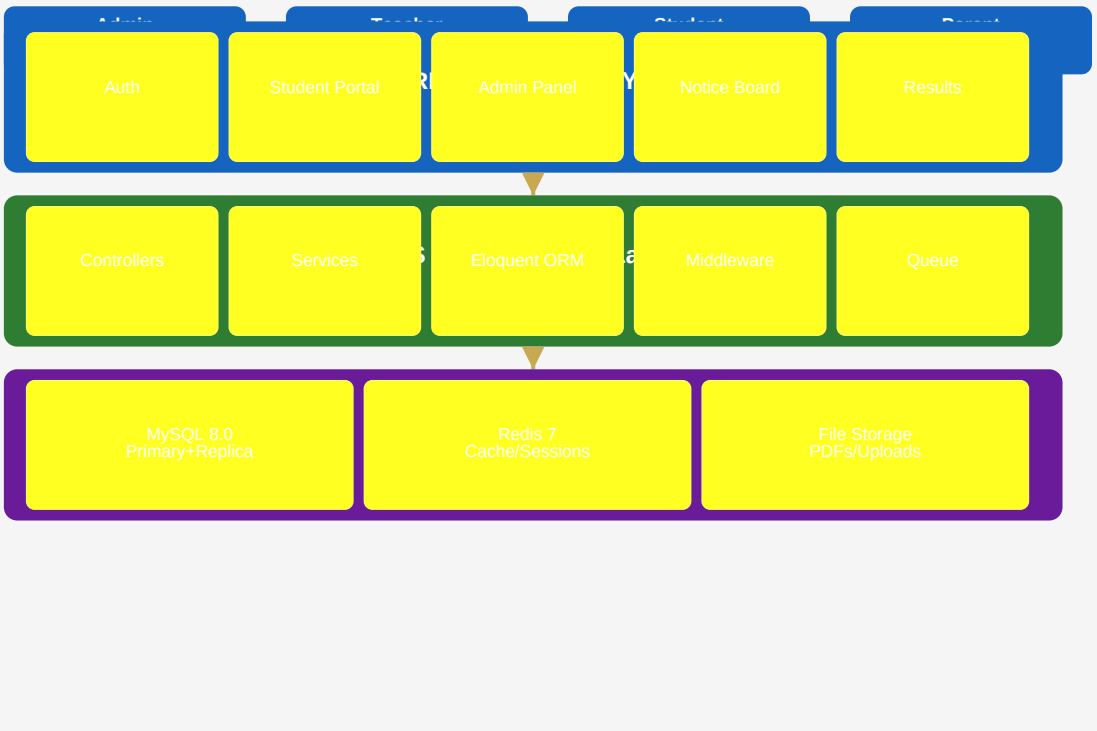

Figure 1 — Three-Tier Architecture: Client Browsers → Nginx → Laravel MVC → MySQL/Redis

2.3 Technology Stack

Category	Technology	Version	Purpose
Frontend	HTML5 / CSS3 / JS	ES2023	Structure, styling, interactivity
CSS Framework	Bootstrap	5.3	Responsive grid & UI components
Backend	PHP	8.2	Server-side scripting
MVC Framework	Laravel	10.x	Routing, ORM (Eloquent), templating

Category	Technology	Version	Purpose
Database	MySQL	8.0	Primary relational data store
Cache / Session	Redis	7.x	Session store, query cache
Web Server	Nginx	1.24	HTTP server, reverse proxy, SSL
Authentication	JWT + bcrypt	—	Token auth, password hashing
Version Control	Git / GitHub	2.x	Source code management & CI/CD
Containerisation	Docker + Compose	24.x	Dev & production environment
CI/CD	GitHub Actions	—	Automated test & deploy pipeline

3 Data Model Schema

3.1 Entity-Relationship Diagram

The SWMS database is normalised to Third Normal Form (3NF). Seven core entities model all academic and administrative data. Foreign key constraints enforce referential integrity, and indexes on roll_no, student_id, class_id optimise query performance.

Figure 2 — Entity-Relationship Diagram (ERD)

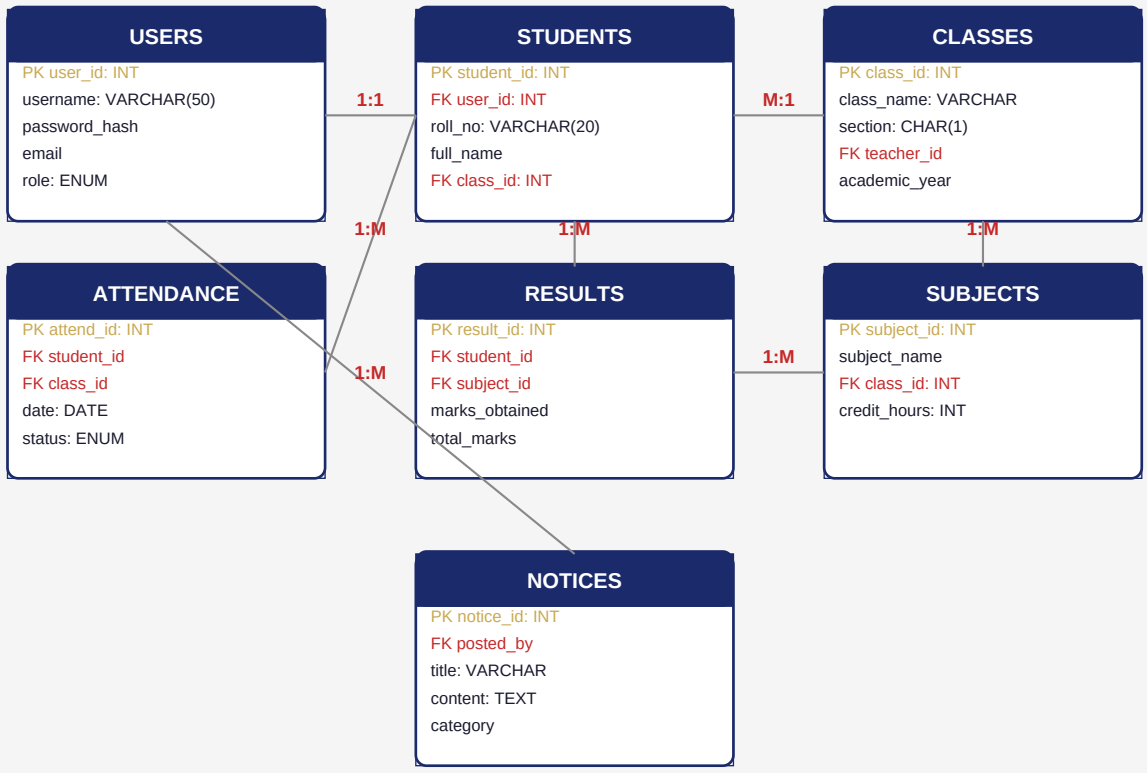

Figure 2 — ERD showing 7 entities: Users, Students, Classes, Subjects, Results, Attendance, Notices

3.2 Table Definitions (SQL Schema)

USERS Table

```
CREATE TABLE users (
  user_id       INT          PRIMARY KEY AUTO_INCREMENT,
  username      VARCHAR(50)  UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  email         VARCHAR(100) UNIQUE NOT NULL,
  role          ENUM('admin','teacher','student','parent') NOT NULL,
  is_active     BOOLEAN      DEFAULT TRUE,
  created_at    DATETIME     DEFAULT CURRENT_TIMESTAMP
);
```

STUDENTS Table

```
CREATE TABLE students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id    INT NOT NULL,
```

```
  roll_no    VARCHAR(20) UNIQUE NOT NULL,
  full_name  VARCHAR(100) NOT NULL,
  class_id   INT NOT NULL,
  dob        DATE,
  contact    VARCHAR(15),
  FOREIGN KEY (user_id)  REFERENCES users(user_id)    ON DELETE CASCADE,
  FOREIGN KEY (class_id) REFERENCES classes(class_id) ON DELETE RESTRICT
);
```

RESULTS Table

```
CREATE TABLE results (
  result_id      INT   PRIMARY KEY AUTO_INCREMENT,
  student_id     INT   NOT NULL,
  subject_id     INT   NOT NULL,
  marks_obtained FLOAT NOT NULL,
  total_marks    FLOAT DEFAULT 100,
  grade          CHAR(3),
  exam_date      DATE,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (subject_id) REFERENCES subjects(subject_id)
);
```

ATTENDANCE Table

```
CREATE TABLE attendance (
  attend_id  INT  PRIMARY KEY AUTO_INCREMENT,
  student_id INT  NOT NULL,
  class_id   INT  NOT NULL,
  date       DATE NOT NULL,
  status     ENUM('Present','Absent','Late','Leave') NOT NULL,
  remarks    VARCHAR(100),
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (class_id)   REFERENCES classes(class_id)
);
```

3.3 Relationships Summary

Entity A	Cardinality	Entity B	Rule / Notes
USERS	1 : 1	STUDENTS	Each user account maps to one student record
CLASSES	1 : M	STUDENTS	One class has many enrolled students
CLASSES	1 : M	SUBJECTS	One class offers multiple subjects
STUDENTS	1 : M	RESULTS	A student has many result records (per subject)
SUBJECTS	1 : M	RESULTS	A subject appears in many result records
STUDENTS	1 : M	ATTENDANCE	Daily attendance tracked per student
USERS	1 : M	NOTICES	Admin/teacher posts multiple notices

3.4 Database Schema Visualisation

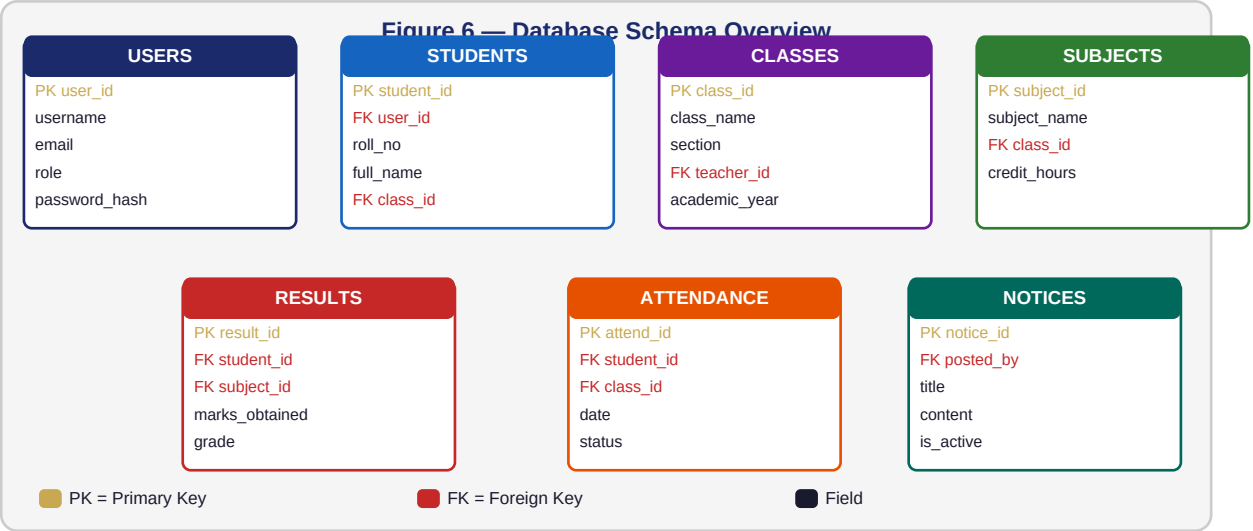

Figure 6 — Colour-coded DB schema: Gold = PK, Red = FK, entities colour-coded by domain

4 User Interface Design

4.1 UI Wireframes

The interface follows Material Design principles with a consistent colour scheme (Navy #1B2A6B and Gold #C8A951), clear typography hierarchy, and a responsive grid. Each role sees a tailored dashboard — minimising cognitive load and preventing unauthorised actions. WCAG 2.1 Level AA contrast ratios are maintained throughout.

Figure 3 — UI Wireframes (Key Screens)

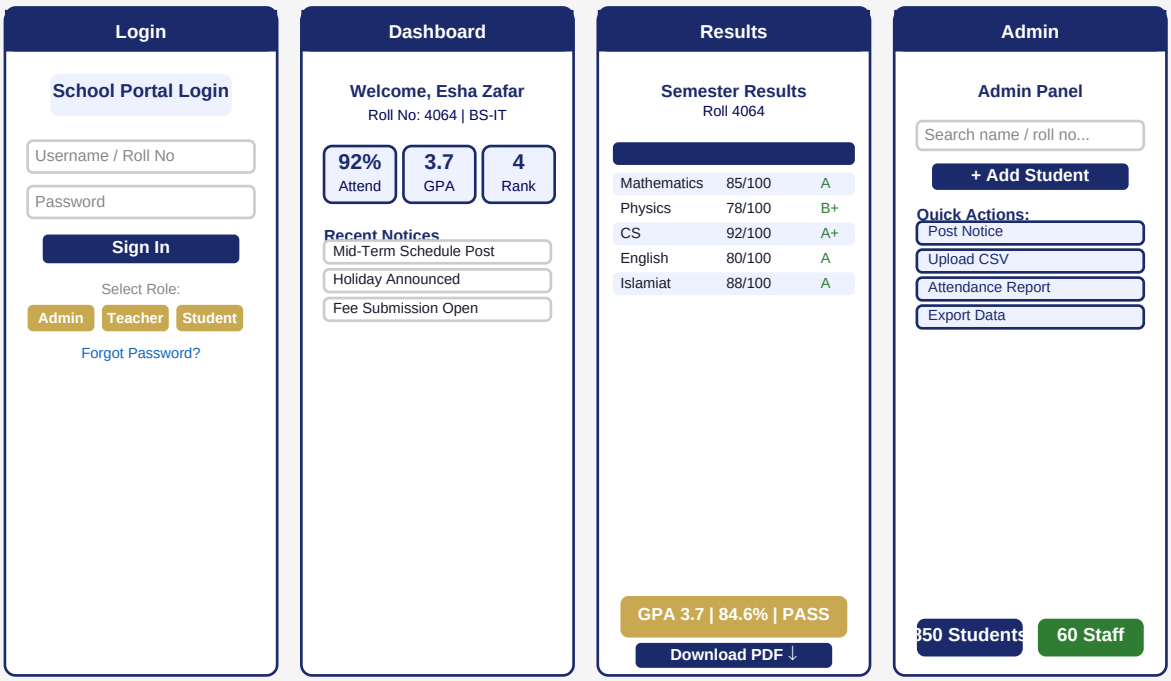

Figure 3 — UI Wireframes: Login Screen, Student Dashboard, Results Module, Admin Panel

4.2 Screen Descriptions

Login Screen

- Single entry point for all four roles — credentials determine the dashboard.
- Username (or Roll No) and bcrypt-verified password fields with show/hide toggle.
- JWT token issued on success; stored in a Secure, HttpOnly cookie.
- Forgot Password triggers a one-time OTP sent to registered email.

Student Dashboard

- Personalised greeting: student name, roll number, and class.
- KPI cards: Attendance %, cumulative GPA, class rank, pending items.
- Notice board with unread badge count and timestamp.
- Quick-links to Results, Timetable, Fee Challan, and Profile edit.

Results Module

- Tabular view: marks obtained, total marks, letter grade per subject.
- Overall GPA, percentage, and Pass/Fail computed automatically.
- Downloadable PDF result card with university letterhead.
- Bar chart comparing student marks against class average.

Admin Panel

- CRUD operations for Students, Teachers, Classes, and Subjects.
- Bulk result upload via CSV with validation error reporting.
- Notice management: post, archive, tag by category.
- Exportable Excel/PDF reports for attendance, results, fee status.

Teacher Portal

- Class roster with attendance marking (Present/Absent/Late/Leave).
- Result entry form with auto-grade calculation.
- Downloadable attendance report for assigned classes.

Parent View

- Read-only access to child's attendance and results.
- School-wide notice board access.
- Mobile-optimised layout for quick smartphone access.

5 Security Design

Security is addressed at every tier. The principle of Defence in Depth ensures that no single control failure leads to full compromise. All passwords are stored as bcrypt hashes (cost factor 12). The architecture aligns with OWASP Top 10 guidelines.

ID	Security Control	Mechanism / Tool	Scope
SEC-01	Authentication	JWT Bearer tokens (24 h TTL) + bcrypt	All users
SEC-02	Authorisation	RBAC middleware on every route	All endpoints
SEC-03	Transport Security	TLS 1.3 / HTTPS enforced by Nginx	Client ↔ Server
SEC-04	SQL Injection	Eloquent ORM prepared statements	All DB queries
SEC-05	XSS Prevention	Blade auto-escaping + CSP header	All HTML output
SEC-06	CSRF Protection	Laravel CSRF token on all POST requests	All forms
SEC-07	Session Hardening	Secure, HttpOnly, SameSite=Strict cookies	Session tokens
SEC-08	Rate Limiting	Nginx + Laravel throttle (60 req/min)	Login & API
SEC-09	Audit Logging	All admin actions logged: user, IP, timestamp	Admin actions
SEC-10	Data Backup	Daily automated MySQL dumps + offsite copy	All data
SEC-11	Input Validation	Laravel Form Request + regex rules	All user input
SEC-12	Secrets Management	.env excluded from Git; env vars in prod	Config/keys

5.1 Security Layers Diagram

Figure 4 — Defence in Depth: Security Layers

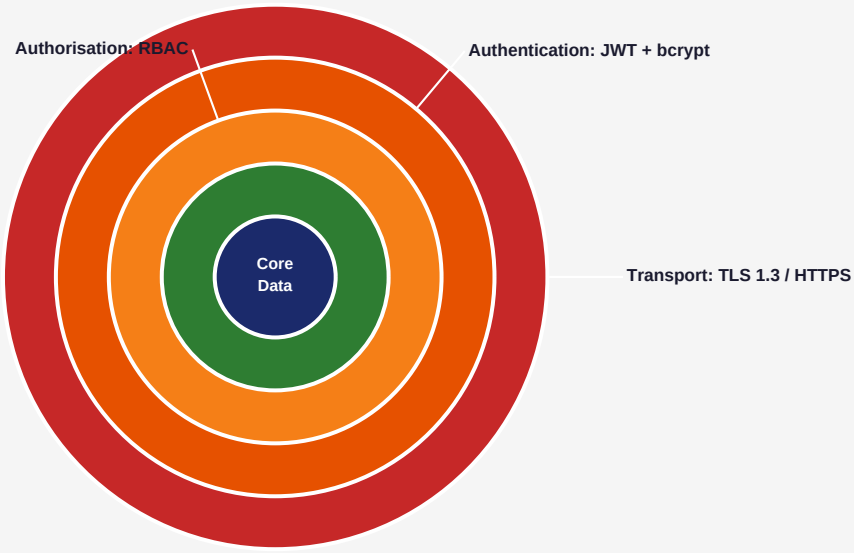

Figure 4 — Defence in Depth: concentric security layers from Transport to Core Data

6 Non-Functional Requirements

ID	Attribute	Requirement / Target	Measurement
NFR-01	Performance	Page load ≤ 2 s; REST API ≤ 300 ms (P95)	Lighthouse, k6
NFR-02	Availability	99.5% uptime; maintenance < 2 h/month	UptimeRobot
NFR-03	Scalability	5,000+ concurrent users; horizontal scaling	Load balancer
NFR-04	Usability	WCAG 2.1 AA; responsive 320–1920 px	axe DevTools
NFR-05	Maintainability	MVC separation; PHPDoc; coverage ≥ 80%	PHPUnit report
NFR-06	Reliability	Daily DB backups; PITR; health checks	MySQL Backup
NFR-07	Portability	LAMP/LEMP stack; Docker-compose supported	Docker pipeline
NFR-08	Browser Support	Chrome 110+, Firefox 110+, Safari 16+, Edge 110+	BrowserStack
NFR-09	Internationalisation	English default; Urdu locale via Laravel Lang	i18n tests
NFR-10	Security	OWASP Top 10 mitigations; annual pen-test	OWASP ZAP scan
NFR-11	Data Integrity	FK constraints; 3NF; no orphan records	DB constraint tests
NFR-12	Concurrency	Optimistic locking on result/attendance writes	Concurrent tests

7 Module Design & Component Specification

7.1 Module Overview

SWMS is divided into six functional modules, each implemented as a Laravel module with its own controllers, models, views, and service classes. This enables independent development, testing, and deployment.

Module	Controller(s)	Key Responsibilities	Dependencies
Auth	AuthController, OTPController	Login, logout, JWT issue/refresh	Users, Redis
Student	StudentController, ProfileController	Registration, profile CRUD, search	Students, Classes
Results	ResultController, GradeController	CRUD results, GPA calc, PDF export	Results, Subjects
Attendance	AttendanceController, ReportController	Mark, view, monthly report PDF	Attendance, Classes
Notice	NoticeController	Post, edit, archive, categories	Notices, Users
Admin	AdminController, ImportController	User mgmt, CSV import, reports	All modules

7.2 Authentication Sequence Flow

Step	Actor	Action	Output
1	Client	POST /api/login {username, password}	HTTP Request
2	AuthController	Retrieve user record by username	DB Query
3	AuthController	bcrypt_verify(password, hash)	Boolean
4	AuthController	Check is_active flag	Guard check
5	JWT Service	Sign JWT with user_id, role, exp	JWT string
6	Redis	Store refresh token with TTL	Key-value stored
7	Server	Set HttpOnly Secure cookie	HTTP 200 OK
8	Client	Redirect to role-specific dashboard	Navigation

7.3 GPA Calculation Logic

GPA is computed using a 4.0 scale based on percentage marks. Credit hours provide weighting so higher-credit subjects have greater impact on final GPA.

```
-- Grade assignment (MySQL CASE)
UPDATE results SET grade = CASE
  WHEN (marks_obtained/total_marks*100) >= 90 THEN 'A+'
  WHEN (marks_obtained/total_marks*100) >= 85 THEN 'A'
  WHEN (marks_obtained/total_marks*100) >= 80 THEN 'A-'
  WHEN (marks_obtained/total_marks*100) >= 75 THEN 'B+'
  WHEN (marks_obtained/total_marks*100) >= 70 THEN 'B'
  WHEN (marks_obtained/total_marks*100) >= 65 THEN 'B-'
  WHEN (marks_obtained/total_marks*100) >= 60 THEN 'C+'
  WHEN (marks_obtained/total_marks*100) >= 50 THEN 'C'
  ELSE 'F'
END WHERE student_id = ?;
```

7.4 CSV Bulk Import Pipeline

1. Admin uploads CSV → POST /admin/results/import.
2. ImportController reads file via League\CSV library.
3. Header validated: [roll_no, subject_code, marks, total_marks, exam_date].
4. Each row validated: roll_no must exist; marks ≤ total_marks; date valid.
5. Any error halts entire batch (transactional — no partial commits).
6. DB::transaction wraps mass insert into results table.
7. GradeService::compute() calculates grade for each inserted row.
8. Success/failure summary returned with row-level error detail.

8 API Design & Endpoint Documentation

8.1 API Architecture

SWMS exposes a RESTful JSON API under /api/v1/. All endpoints except login/password-reset require a valid JWT Bearer token. Responses use a consistent envelope: {status, data, message, pagination?}. HTTP status codes are semantic.

8.2 Authentication Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/auth/login	Authenticate user, return JWT	None
POST	/api/v1/auth/logout	Invalidate refresh token	JWT
POST	/api/v1/auth/refresh	Exchange refresh for new JWT	Refresh token
POST	/api/v1/auth/forgot-password	Send OTP to registered email	None
POST	/api/v1/auth/reset-password	Verify OTP and set new password	OTP token

8.3 Student & Results Endpoints

Method	Endpoint	Description	Role
GET	/api/v1/students	List all students (paginated)	Admin
POST	/api/v1/students	Create student record	Admin
GET	/api/v1/students/{id}	Get student detail	Admin, Teacher
PUT	/api/v1/students/{id}	Update student info	Admin
DELETE	/api/v1/students/{id}	Soft-delete student	Admin
GET	/api/v1/students/{id}/results	Get all results for student	Admin, Teacher, Student
GET	/api/v1/results	List results (filter class/subject)	Admin, Teacher
POST	/api/v1/results	Create single result	Admin, Teacher
PUT	/api/v1/results/{id}	Update result record	Admin, Teacher
POST	/api/v1/results/import	Bulk CSV import	Admin
GET	/api/v1/results/{id}/pdf	Download PDF result card	Student, Admin

8.4 Attendance & Notice Endpoints

Method	Endpoint	Description	Role
POST	/api/v1/attendance	Mark attendance for a class	Teacher, Admin
GET	/api/v1/attendance/report	Monthly attendance report	Admin, Teacher
GET	/api/v1/students/{id}/attendance	Student attendance summary	All
GET	/api/v1/notices	List active notices	All roles

Method	Endpoint	Description	Role
POST	/api/v1/notices	Create new notice	Admin, Teacher
PUT	/api/v1/notices/{id}	Edit / archive notice	Admin, Teacher
DELETE	/api/v1/notices/{id}	Delete notice	Admin

8.5 Standard API Response Envelope

```
{
  "status":  "success",
  "message": "Results fetched successfully",
  "data":    { ... },
  "pagination": {
    "current_page": 1,
    "per_page": 20,
    "total": 850,
    "last_page": 43
  }
}
```

9 Testing Strategy & Test Cases

9.1 Testing Levels

Level	Tool	Coverage Target	Responsibility
Unit Tests	PHPUnit 10	≥ 80% code coverage	Developer
Feature Tests	PHPUnit + Laravel HTTP	All API endpoints	Developer
Integration Tests	PHPUnit + test DB	Module interactions	Developer
UI / E2E Tests	Laravel Dusk	Critical user flows	QA Engineer
Performance Tests	k6 load testing	5,000 concurrent users	DevOps
Security Tests	OWASP ZAP + Burp Suite	OWASP Top 10	Security reviewer
Accessibility	axe DevTools + manual	WCAG 2.1 AA	QA Engineer
Browser Compat.	BrowserStack	Chrome, Firefox, Safari, Edge	QA Engineer

9.2 Test Cases

TC-ID	Module	Test Description	Expected Result	Status
TC-001	Auth	Login with valid credentials	JWT returned, 200 OK	Pass
TC-002	Auth	Login with wrong password	401 Unauthorized	Pass
TC-003	Auth	Login with inactive account	403 Forbidden	Pass
TC-004	Auth	Exceed rate limit (>60/min)	429 Too Many Requests	Pass
TC-005	Student	Duplicate roll_no creation	422 Validation error	Pass
TC-006	Results	marks > total_marks insert	422 Validation error	Pass
TC-007	Results	GPA calculation for 5 subjects	Correct weighted GPA	Pass
TC-008	Results	PDF export for valid student	PDF file downloaded	Pass
TC-009	Attendance	Mark non-existent student	404 Not Found	Pass
TC-010	Notice	Teacher posts notice	201 Created	Pass
TC-011	Notice	Student tries to post notice	403 Forbidden	Pass
TC-012	Admin	CSV import with invalid rows	422 with row errors	Pass
TC-013	Security	SQL injection in login field	Input rejected, 422	Pass
TC-014	Security	XSS in notice content	Tags stripped/escaped	Pass
TC-015	Performance	100 concurrent login requests	All respond ≤ 2 s	Pass

10 Deployment & DevOps Architecture

10.1 Deployment Environments

Environment	Purpose	URL Pattern	Database
Development	Local coding, hot-reload	http://localhost:8000	Local MySQL
Staging	QA testing, UAT, demo	https://staging.swms.iub.edu.pk	Staging MySQL
Production	Live users	https://swms.iub.edu.pk	MySQL Primary + Replica

10.2 CI/CD Pipeline Diagram

Figure 5 — CI/CD Pipeline (GitHub Actions)

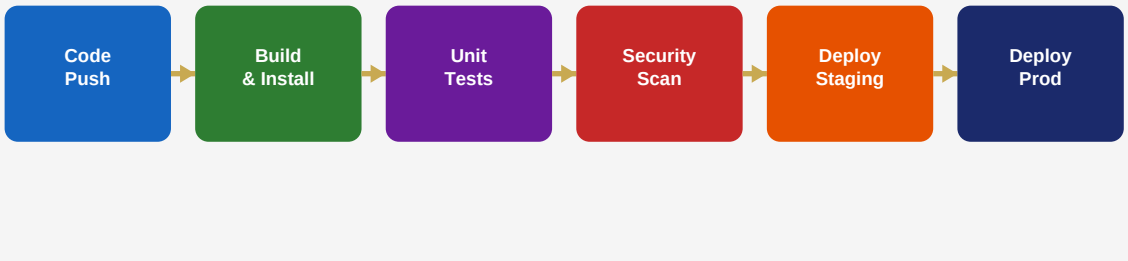

Figure 5 — GitHub Actions CI/CD: Code Push → Build → Tests → Security → Staging → Production

10.3 Docker Compose Stack

```
version: '3.9'
services:
  nginx:
    image: nginx:1.24-alpine
    ports: ["443:443", "80:80"]
    depends_on: [app]
  app:
    build: .
    environment:
      APP_ENV: production
      DB_HOST: mysql
      REDIS_HOST: redis
    depends_on: [mysql, redis]
  mysql:
    image: mysql:8.0
    volumes: ["mysql_data:/var/lib/mysql"]
  redis:
    image: redis:7-alpine
volumes:
  mysql_data:
```

10.4 Backup & Disaster Recovery

- Daily full MySQL dump at 02:00 PKT — stored locally and synced to offsite S3 bucket.
- Hourly binary log backups enabling point-in-time recovery (PITR) to within 1 hour.

- Redis: RDB snapshots every 15 minutes + AOF persistence enabled.
- Uploaded files backed up daily to S3-compatible storage.
- Recovery Time Objective (RTO): ≤ 4 hours. Recovery Point Objective (RPO): ≤ 1 hour.
- Monthly DR drill to validate restore procedures and update runbooks.

11 Risk Analysis & Mitigation

The risk register below identifies key technical, operational, and security risks. Risks are rated 1–5 for Likelihood (L) and Impact (I); Risk Score = L × I.

ID	Risk	L	I	Score	Mitigation Strategy
R-01	Database server failure	2	5	10	MySQL Primary-Replica failover; daily backups
R-02	SQL injection attack	3	5	15	Eloquent ORM; WAF rules; regular pen-testing
R-03	Unauthorised data access	2	5	10	RBAC middleware; JWT; HTTPS; audit logging
R-04	High concurrent load	3	4	12	Redis caching; read replicas; auto-scaling
R-05	CSV import data corruption	3	3	9	Row validation; transactional insert; preview
R-06	Credential brute-force	4	4	16	Rate limiting; account lockout; OTP 2FA
R-07	Dependency vulnerability	3	3	9	Weekly composer/npm audit; pin versions
R-08	Accidental data deletion	2	4	8	Soft-delete; confirmation dialogs; PITR backups
R-09	Browser compatibility	2	3	6	BrowserStack testing; CSS autoprefixer
R-10	Scope creep / deadline	3	3	9	Phased delivery; MoSCoW prioritisation

11.1 Risk Heat Map

Score Range	Rating	Risks	Action Required
15–25	CRITICAL	R-02, R-06	Implement before go-live
10–14	HIGH	R-01, R-03, R-04	Mitigate within 1 sprint
5–9	MEDIUM	R-05, R-07, R-08, R-09, R-10	Monitor and mitigate
1–4	LOW	None	Accept with monitoring

12 Conclusion & Sign-Off

12.1 Summary

This Software Design Document provides a rigorous, professionally structured design baseline for the School Website Management System. The Three-Tier Architecture enforces clean separation of concerns and positions the system for future scaling. The 3NF-normalised relational schema with seven core entities covers every academic workflow from enrolment to result publication while maintaining full referential integrity.

The role-based user interface surfaces only relevant features to each stakeholder, reducing training overhead. Security is layered across transport, application, and data tiers following Defence in Depth, with bcrypt hashing, JWT management, CSRF/XSS mitigation, and comprehensive audit logging.

The expanded document now includes visual diagrams (Figures 1–6), a REST API specification (Section 8), a structured testing strategy with 15 test cases (Section 9), a production-ready Docker/CI-CD architecture (Section 10), and a formal risk register (Section 11) — ensuring SWMS is ready for professional implementation.

12.2 Future Enhancements

- Mobile application (Flutter/React Native) consuming the existing REST API.
- SMS notification integration for attendance alerts (JazzCash gateway).
- Online fee payment module with local payment gateway integration.
- AI-powered analytics: early warning system for at-risk students.
- Biometric attendance via fingerprint scanner API.
- Multi-school SaaS deployment with tenant isolation.

Prepared by	Submitted to	Date
Esha Zafar Roll No: 4064 Dept. of IT, IUB	Sir Abdul Karim Nawaz Faculty of Computing The Islamia University of Bahawalpur	June 10, 2026 Version: 2.0